Bayesian Neural Networks and Monte Carlo Dropout

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

University of Ottawa

November 9, 2026

Roadmap

1. Likelihood and Output Heads
2. MCMC for BNNs
3. Variational Inference: Bayes by Backprop
4. MC Dropout: Training and Inference
5. Why VI when MC Dropout is so Simple?
6. Diagnostics
7. Summary

What This Lecture Covers (Implementation Focus)

How methods are implemented:

1. Likelihood and output heads (heteroscedastic loss)
2. MCMC for BNNs: SGLD, HMC — practical algorithms
3. Variational inference: Bayes-by-Backprop in detail
4. MC Dropout: exact training and inference procedure
5. **Why VI when MC Dropout is so simple?** — decision guide

Key references

Gal & Ghahramani (2016)
Blundell et al. (2015)
Kendall & Gal (2017)
Welling & Teh (2011) — SGLD

From Point Estimates to Weight Distributions

Standard (deterministic) NN

1. **Model:** likelihood $p(y \mid x, w)$
2. **Cost:** $\mathcal{L}(w) = -\log p(\mathcal{D} \mid w)$ (adding $-\log p(w)$ gives MAP, e.g. L2 weight decay)
3. **Train:** minimise, $\hat{w} = \arg\min_w \mathcal{L}(w)$
4. **Predict:** plug in, $p(y^* \mid x^*, \hat{w})$

$\hat{w}$ is a **single point** in parameter space.

Bayesian neural network (BNN)

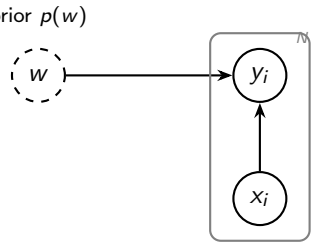
1. **Model:** the *same* likelihood + prior $p(w)$
2. **Cost:** SGLD: $U(w) = -\log p(\mathcal{D} \mid w) - \log p(w)$
   VI: $-\mathrm{ELBO} = \mathbb{E}_q[-\log p(\mathcal{D} \mid w)] + \mathrm{KL}(q \,\|\, p)$
3. **Infer** $p(w \mid \mathcal{D}) \propto p(\mathcal{D} \mid w)\, p(w)$: *sample* $w^{(s)}$ (SGLD) or *fit* $q(w)$ (VI)
4. **Predict:** average, $\frac{1}{S} \sum_s p(y^* \mid x^*, w^{(s)})$

$p(w \mid \mathcal{D})$ is a **distribution** over parameters.

Same cost, different use

The SGLD cost $U(w)$ *is* the MAP cost: MAP minimises it, SGLD samples from it. $p(w \mid \mathcal{D})$ encodes **which model configurations are consistent with the data**: few training samples nearby $\Rightarrow$ many plausible w $\Rightarrow$ wide predictive distribution.

Graphical Model View of a BNN



Latent (dashed): w — unknown weights
Observed (solid): (x_i, y_i) — data
Plate: replicated over N examples

Bayes' theorem for BNNs

$$\underbrace{p(w \mid \mathcal{D})}_{\text{posterior}} \propto \underbrace{p(\mathcal{D} \mid w)}_{\text{likelihood}} \cdot \underbrace{p(w)}_{\text{prior}}$$

Likelihood models

Regression: $y_i \mid x_i, w \sim \mathcal{N}\big(f_w(x_i),\, \sigma^2\big)$

Classification: $y_i \mid x_i, w \sim \mathrm{Cat}(\mathrm{softmax}(f_w(x_i)))$

Common priors

$p(w) = \prod_j \mathcal{N}(w_j; 0, \tau^2)$
(i.i.d. zero-mean Gaussian, equivalent to L2 regularization at MAP)

Posterior Predictive = Bayesian Model Averaging

Posterior predictive distribution

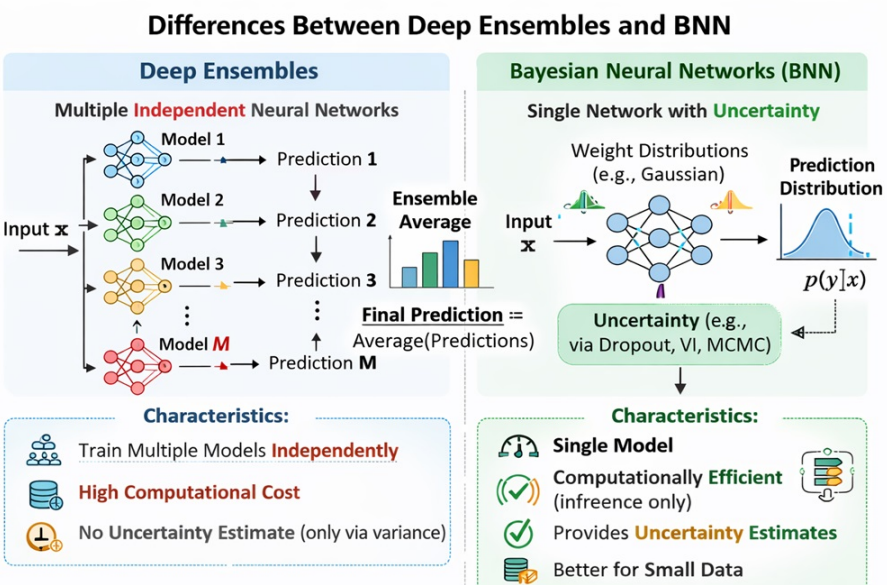
$$p(y^* \mid x^*, \mathcal{D}) = \int p(y^* \mid x^*, w)\, p(w \mid \mathcal{D})\, \mathrm{d}w \approx \frac{1}{S} \sum_{s=1}^{S} p(y^* \mid x^*, w^{(s)}), \quad w^{(s)} \sim p(w \mid \mathcal{D})$$

Intuition: each $w^{(s)}$ defines a different network function $f_{w^{(s)}}$. The BNN *averages over many plausible networks*.

Why this differs from an ensemble

In a BNN the samples $w^{(s)}$ come from the *posterior* (data-informed), not from re-training with different random seeds.

Deep Ensembles vs. BNNs



Why the Posterior is Intractable

The posterior requires normalising the likelihood:

$$p(w \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid w)\, p(w)}{p(\mathcal{D})}, \qquad p(\mathcal{D}) = \int p(\mathcal{D} \mid w)\, p(w)\, \mathrm{d}w.$$

Practical difficulties

- $w \in \mathbb{R}^d$ for $d =$ thousands–billions in modern nets
- The integral $p(\mathcal{D})$ is computationally intractable (high-dimensional, non-Gaussian)
- The posterior is **multi-modal**: many local minima, complex geometry

Three families of approximations

1. **MCMC** (HMC/NUTS, SGLD): stochastically sample $w^{(s)} \sim p(w \mid \mathcal{D})$ — asymptotically exact, expensive
2. **Variational inference**: optimise a tractable $q_\phi(w) \approx p(w \mid \mathcal{D})$ — fast, biased
3. **MC Dropout**: use dropout masks as an implicit variational distribution — cheapest, good baseline

The Likelihood for BNN Regression: Heteroscedastic NLL

Standard regression outputs a single value. A regression head can output *both* mean and log-variance, $(\hat{\mu}_w(x),\ \log \hat{\sigma}^2_w(x))$.

This alone is not a BNN: minimising $\mathcal{L}_{\text{het}}$ gives an **MVE network** (point estimate $\hat{w}$, aleatoric uncertainty only). SGLD, BbB and MC dropout put this NLL *inside* their objective and add a distribution over w.

Heteroscedastic NLL loss

$$\mathcal{L}_{\text{het}} = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{(y_i - \hat{\mu}_w(x_i))^2}{2\,e^{s_i}} + \frac{s_i}{2}\right]$$

where $s_i = \log \hat{\sigma}^2_w(x_i)$.

Why not MSE? MSE ignores aleatoric noise. Large-noise samples should contribute less — the heteroscedastic loss provides this automatic down-weighting.

PyTorch implementation

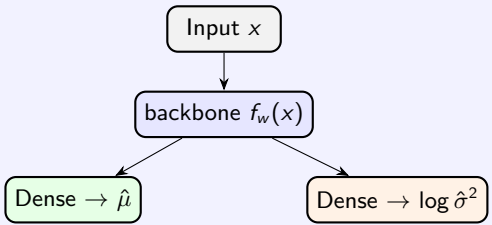
```
def hetero_nll(y, mu, log_var):
    # y, mu, log_var: (B,)
    prec = torch.exp(-log_var)
    return 0.5 * (
        prec * (y - mu)**2
        + log_var
    ).mean()
```

Numerical tip

Predict $\log \hat{\sigma}^2$, not $\hat{\sigma}^2$, to guarantee positivity and avoid gradient explosion.

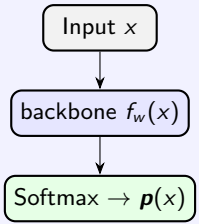
Output Heads (shared by MVE Networks and BNNs)

Regression: dual-head design



Loss = heteroscedastic NLL.

Classification: softmax head



Loss = cross-entropy.
Uncertainty from MC-averaged softmax, not from the architecture.

Where dropout goes

Dropout is placed *inside* the network (after hidden layers). The heads themselves typically do *not* have dropout so that mean and variance outputs are not further masked.

Stochastic Gradient Langevin Dynamics (SGLD)

HMC requires full-batch gradients and is too slow for large nets. **SGLD** injects Gaussian noise into SGD updates to obtain asymptotically correct posterior samples. w_t = the full weight vector, **one value per weight** (as in SGD); the posterior is the set of *saved samples*.

SGLD update rule

$$w_{t+1} = w_t - \frac{\epsilon_t}{2}\nabla_w \tilde{U}(w_t) + \boldsymbol{\eta}_t, \quad \boldsymbol{\eta}_t \sim \mathcal{N}(0,\, \epsilon_t I)$$

where $\tilde{U}(w) = -\frac{N}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} \log p(y_i|x_i, w) - \log p(w)$ is the stochastic potential (negative log-posterior, mini-batch scaled).

Full SGLD algorithm

Initialise w_0; set step schedule $\{\epsilon_t\}$
for $t = 1, 2, \ldots$ **do**
  Sample mini-batch $\mathcal{B} \subset \mathcal{D}$
  Compute stochastic gradient $g_t = \nabla_w \tilde{U}(w_t)$
  Sample $\boldsymbol{\eta}_t \sim \mathcal{N}(0,\, \epsilon_t I)$
  $w_{t+1} \leftarrow w_t - \frac{\epsilon_t}{2} g_t + \boldsymbol{\eta}_t$
  if $t > t_{\text{burn-in}}$ and $t \bmod K = 0$ **then**
    Save w_t as a posterior sample
  end if
end for

Intuition

SGD = gradient descent.
SGLD = SGD + noise injection.

The noise prevents the chain from collapsing to a mode.
The step size schedule $\epsilon_t \to 0$ ensures the chain converges to $p(w|\mathcal{D})$ rather than oscillating.

Practical notes

- Burn-in: discard first $t_{\text{burn-in}}$ samples (chain not yet mixed)
- Thinning: save every K-th sample to reduce correlation
- Step size matters: too large $\Rightarrow$ inaccurate; too small $\Rightarrow$ slow mixing
- Cyclical SGLD (Zhang et al. 2020) improves sample diversity

SGLD: Posterior Predictive and Inference

Samples: one long run; discard the burn-in, keep every K-th $w_t \Rightarrow \{w^{(1)}, \ldots, w^{(S)}\}$. "Converged" = the visited w_t are distributed as $p(w \mid \mathcal{D})$; the weights keep moving.

Each $w^{(s)}$ is a **full network with both heads**: it predicts a mean $\mu_{w^{(s)}}(x^*)$ and a noise variance $\sigma^2_{w^{(s)}}(x^*)$.

Monte Carlo predictive

$$p(y^*|x^*, \mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} p(y^*|x^*, w^{(s)})$$

$$\hat{\mu} = \frac{1}{S}\sum_{s} \mu_{w^{(s)}}(x^*), \quad \hat{\sigma}^2_{\text{ep}} = \frac{1}{S}\sum_{s} \big(\mu_{w^{(s)}}(x^*) - \hat{\mu}\big)^2$$

$$\hat{\sigma}^2_{\text{al}}(x^*) = \frac{1}{S}\sum_{s=1}^{S} \sigma^2_{w^{(s)}}(x^*)$$

- **Epistemic** = spread of the *means* across weight samples: "how much do plausible networks disagree about y^*?"
- **Aleatoric** = average of the predicted *noise variances*: "how noisy are the measurements at x^*?" (next slide)

When to use SGLD over MC Dropout

Use SGLD when:

- The model has *no* dropout layers
- You need a benchmark posterior (e.g. for validating VI)
- Dataset is small-to-medium (fit in GPU memory or fast iteration)
- You need asymptotically correct samples

Drawback:
Storage of S full model checkpoints ($\sim S\times$ model size).

Aleatoric vs. Epistemic: Why the Split Works

With $y^* \mid x^*, w \sim \mathcal{N}\big(\mu_w(x^*), \sigma_w^2(x^*)\big)$ and $w \sim p(w \mid \mathcal{D})$, the law of total variance gives

$$\mathrm{Var}[y^* \mid x^*, \mathcal{D}] = \underbrace{\mathbb{E}_w\big[\sigma_w^2(x^*)\big]}_{\text{aleatoric} \approx \hat{\sigma}^2_{\text{al}}} + \underbrace{\mathrm{Var}_w\big[\mu_w(x^*)\big]}_{\text{epistemic} \approx \hat{\sigma}^2_{\text{ep}}}.$$

Test: what happens with infinite data?

The posterior concentrates on one w^*:

- epistemic $\to 0$ (all samples agree on the mean);
- aleatoric $\to \sigma^2_{w^*}(x^*)$, the true noise variance; it does *not* vanish.

Averaging over samples does not make a term epistemic; *what* is averaged decides.

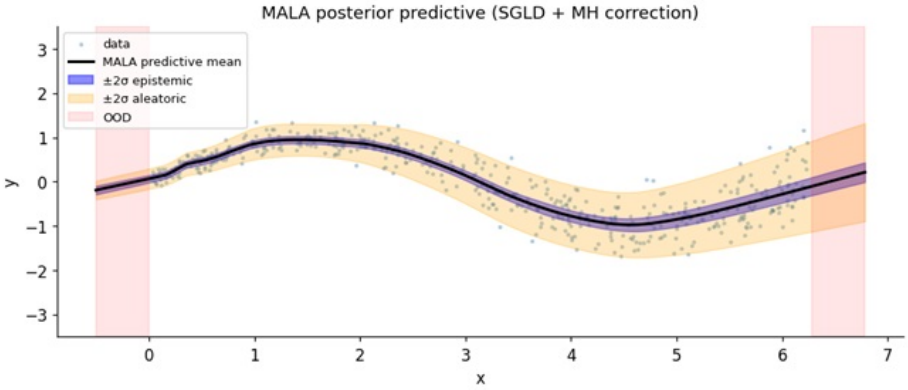
Example at one x^*, $S = 3$

	$s=1$	$s=2$	$s=3$
$\mu_{w^{(s)}}$	2.0	2.4	1.6
$\sigma^2_{w^{(s)}}$	0.25	0.30	0.20

$\hat{\mu} = 2.0$, $\hat{\sigma}^2_{\text{ep}} \approx 0.11$, $\hat{\sigma}^2_{\text{al}} = 0.25$, total ≈ 0.36.

The *spread* of $\sigma^2_{w^{(s)}}$ (0.20–0.30) is epistemic uncertainty *about the noise level*; the formula averages it away. With a fixed-variance likelihood $\mathcal{N}(f_w(x), \sigma^2)$ the aleatoric term is the constant σ^2.

SGLD Posterior Predictive Example



Variational Inference: The Optimisation View

Core idea: replace sampling with optimisation. Find $q_\phi(w)$ that minimises:

$$D_{\mathrm{KL}}(q_\phi(w) \,\|\, p(w|\mathcal{D})) \;\equiv\; \max_{\phi} \; \mathcal{L}(\phi)$$

where

$$\mathcal{L}(\phi) = \underbrace{\mathbb{E}_{w\sim q_\phi}[\log p(\mathcal{D}|w)]}_{\text{expected log-likelihood} \uparrow} - \underbrace{D_{\mathrm{KL}}(q_\phi(w) \,\|\, p(w))}_{\text{KL regulariser} \downarrow}$$

Mean-field Gaussian variational family

$$q_\phi(w) = \prod_j \mathcal{N}(w_j; \mu_j, \sigma_j^2)$$

Each weight w_j has its own learnable (μ_j, σ_j).

Number of parameters: $2d$ instead of d.

Reparameterisation: $w_j = \mu_j + \sigma_j \varepsilon_j, \quad \varepsilon_j \sim \mathcal{N}(0, 1)$

Analytic KL for Gaussian families

$$D_{\mathrm{KL}}(q_\phi \,\|\, p) = \frac{1}{2} \sum_j \left[\frac{\mu_j^2 + \sigma_j^2}{\tau^2} - 1 - \log \frac{\sigma_j^2}{\tau^2} \right]$$

This term is differentiable: gradients flow directly from the KL to (μ_j, σ_j).

Bayes by Backprop (Blundell et al., 2015)

Algorithm

Store mean $\boldsymbol{\mu}$ and (softplus-parameterised) std $\boldsymbol{\sigma} = \mathrm{softplus}(\boldsymbol{\rho})$ for every weight.
Per mini-batch $\mathcal{B}$:

1. Sample $\varepsilon \sim \mathcal{N}(0,\, I)$; compute $w = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon$
2. Estimate the stochastic ELBO:

$$\widehat{\mathcal{L}} = \frac{N}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \log p(y_i \mid x_i, w) - D_{\mathrm{KL}}(q_\phi(w) \,\|\, p(w))$$

3. Gradient step on $(\boldsymbol{\mu}, \boldsymbol{\rho})$ via backprop

Bayes by Backprop: Full Training Algorithm

BbB training algorithm (Blundell 2015)

1: **Initialise:** $\boldsymbol{\mu}$ and $\boldsymbol{\rho}$ ($\sigma = \text{softplus}(\rho) \approx 0.05$)
2: **for** each mini-batch $\mathcal{B}$ of size B **do**
3: **Sample** noise: $\varepsilon \sim \mathcal{N}(0, I)$
4: **Sample** weights: $w = \boldsymbol{\mu} + \text{softplus}(\boldsymbol{\rho}) \odot \varepsilon$
5: **Forward pass** with w, compute predictions
6: **Compute stochastic ELBO:**

$$\widehat{\mathcal{L}} = \frac{N}{B} \sum_{i \in \mathcal{B}} \log p(y_i | x_i, w) - D_{\mathrm{KL}}(q_\phi \,\|\, p)$$

7: **Backprop** through $\boldsymbol{\mu}$ and $\boldsymbol{\rho}$ (reparameterisation)
8: **Adam/SGD** update on $(\boldsymbol{\mu}, \boldsymbol{\rho})$
9: **end for**
10: **Return:** $q_\phi(w) = \prod_j \mathcal{N}(w_j; \mu_j, \text{softplus}(\rho_j)^2)$

PyTorch BbB layer (# = step)

```
class BayesLinear(nn.Module):
  def __init__(self, in_f, out_f,
               prior_std=1.0):
    super().__init__()
    # 1: mu as in nn.Linear, sigma~0.05
    self.mu = nn.Parameter(
        torch.empty(out_f, in_f))
    nn.init.kaiming_uniform_(
        self.mu, a=math.sqrt(5))
    self.rho = nn.Parameter(
        torch.full((out_f,in_f),-3.))
    self.prior_std = prior_std

  def forward(self, x):
    sigma = F.softplus(self.rho)
    eps = torch.randn_like(sigma) # 3
    w = self.mu + sigma * eps     # 4
    s2 = self.prior_std**2  # 6: KL
    kl = 0.5*((self.mu**2+sigma**2)/s2
      - 1 - torch.log(sigma**2/s2)).sum()
    return F.linear(x, w), kl     # 5
```

Steps 2, 6–8 and 10: training loop on the next slide.

Bayes by Backprop in PyTorch: Model and Training Loop

Model and training loop (# = step)

```
class BayesMLP(nn.Module):
  def __init__(self, d_in, d_h):
    super().__init__()
    self.l1 = BayesLinear(d_in, d_h)
    self.l2 = BayesLinear(d_h, 2)    # (mu, log_var)

  def forward(self, x):              # new eps, w each call
    h, kl1 = self.l1(x)
    out, kl2 = self.l2(torch.relu(h))
    return out[:,0], out[:,1], kl1 + kl2

model = BayesMLP(d_in, 50)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
N = len(train_set)

for epoch in range(n_epochs):
  for xb, yb in loader:              # 2
    mu, log_var, kl = model(xb)      # 3-5
    nll = hetero_nll(yb, mu, log_var)
    loss = nll + beta * kl / N       # 6: -ELBO / N
    opt.zero_grad()
    loss.backward()                  # 7
    opt.step()                       # 8
# 10: q(w) = trained (mu, rho) of every layer
```

Why `loss = nll + kl/N`

Step 6 divided by N:

$$-\frac{\widehat{\mathcal{L}}}{N} = \frac{1}{B}\sum_{i\in\mathcal{B}}\big[-\log p(y_i \mid x_i, w)\big] + \frac{\mathrm{KL}}{N}$$

The first term is `hetero_nll` (up to a constant). Dividing by N only rescales the gradient.

Notes

`beta`: KL annealing, $0 \to 1$ (next slide); $\beta = 1$ is the true ELBO.

The KL counts once per data set, so it is divided by N, not B.

BbB: Inference and Practical Pitfalls

Test-time inference

1: **for** $s = 1, \ldots, S$ **do**
2: Sample $\varepsilon \sim \mathcal{N}(0, I)$
3: $w^{(s)} = \boldsymbol{\mu} + \text{softplus}(\boldsymbol{\rho}) \odot \varepsilon$
4: $\hat{y}^{(s)}, \hat{\sigma}^{2,(s)} = f_{w^{(s)}}(x^*)$
5: **end for**
6: $\hat{\mu} = \frac{1}{S}\sum_s \hat{y}^{(s)}$, $\hat{\sigma}^2_{\text{al}} = \frac{1}{S}\sum_s \hat{\sigma}^{2,(s)}$
7: $\hat{\sigma}^2_{\text{ep}} = \frac{1}{S}\sum_s (\hat{y}^{(s)} - \hat{\mu})^2$
8: $\hat{\sigma}^2_{\text{total}} = \hat{\sigma}^2_{\text{ep}} + \hat{\sigma}^2_{\text{al}}$

Fixed-variance likelihood: $\hat{\sigma}^2_{\text{al}} = \sigma^2$.

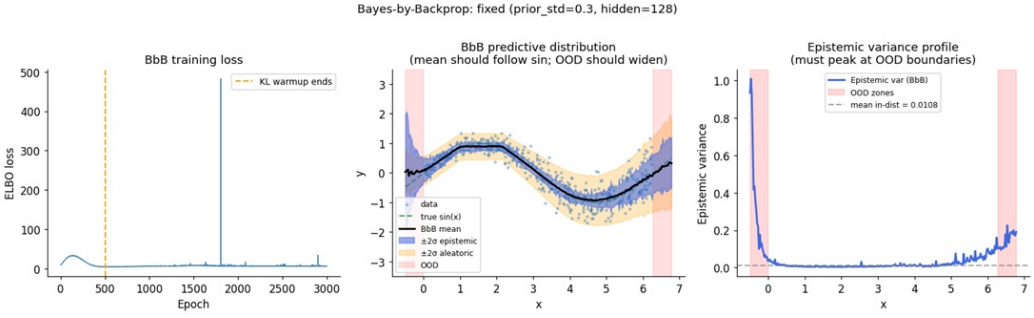
KL cost annealing trick

Early on, the KL can pull every weight back to the prior ($q(w) \approx p(w)$) before the model fits the data. **Fix:** use $\beta \cdot \text{KL}$, with $\beta : 0 \to 1$ over the first T_{warm} epochs.

Common pitfalls

1. **Double parameters**: two numbers per weight (μ_j, ρ_j), so twice the memory, and twice the optimiser state.
2. **High gradient variance**: one weight sample is shared by the whole mini-batch, so its noise does not average out. *Local reparameterisation* samples each pre-activation directly:
   $a_k = \sum_i \mu_{ki} x_i + \sqrt{\sum_i \sigma^2_{ki} x_i^2}\ \varepsilon_k$, with a fresh ε_k per unit and per example.
3. **Prior choice**: the KL pulls $\mu_j \to 0$ (strength $1/\tau^2$) and $\sigma_j \to \tau$. Too small $\tau \Rightarrow$ strong weight decay, underfitting. Too large $\Rightarrow$ almost no pull on μ_j, while σ_j is still pushed up. Matters most with little data.
4. **Initialisation**: $\rho = -3$ so $\sigma \approx 0.05$; initialise μ like a standard layer, not at zero.

Variational Inference Example

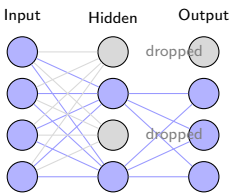


Standard Dropout (Srivastava et al., 2014) — Recap

During training: randomly zero each neuron with probability p:

$$\tilde{h}_j = h_j \cdot m_j, \quad m_j \sim \text{Bernoulli}(1-p)$$

$$m_j = \begin{cases} 1 & \text{with probability } 1-p \quad \text{(keep neuron)}, \\ 0 & \text{with probability } p \quad \text{(drop neuron)}. \end{cases}$$



Training:

- Each forward pass uses a different mask $\boldsymbol{m}^{(s)}$
- Acts as a regulariser (implicit ensemble)

Standard test time:

- Dropout *disabled*
- Weights scaled by $(1-p)$
- Returns a **single** prediction

MC Dropout: Keep Dropout On at Test Time

Key idea (Gal & Ghahramani, 2016): leave dropout *enabled* during inference and run T stochastic forward passes.

MC Dropout inference

For test input x^*, sample T masks $\boldsymbol{m}^{(1)}, \ldots, \boldsymbol{m}^{(T)}$:

$$\hat{y}^{(t)} = f_{\hat{w}, \boldsymbol{m}^{(t)}}(x^*), \quad t = 1, \ldots, T$$

- **Predictive mean:** $\hat{\mu}(x^*) = \frac{1}{T} \sum_{t=1}^{T} \hat{y}^{(t)}$
- **Epistemic variance:** $\hat{\sigma}^2_{\text{ep}}(x^*) = \frac{1}{T} \sum_{t=1}^{T} (\hat{y}^{(t)} - \hat{\mu})^2$
- **Total variance** (if model also outputs $\hat{\sigma}^2_{al}$):

$$\hat{\sigma}^2_{\text{total}} = \hat{\sigma}^2_{\text{ep}} + \frac{1}{T} \sum_{t=1}^{T} \hat{\sigma}^{2,(t)}_{\text{al}}$$

Variational Interpretation of MC Dropout

Gal & Ghahramani (2016): dropout training approximately minimises a **variational objective**.

Dropout as VI (key result)

Let $\boldsymbol{m} \sim \text{Bernoulli}(1-p)^d$ be the dropout mask. Define the variational distribution:

$$q(w) = q(\boldsymbol{M}, \boldsymbol{\mu}), \qquad W_l = \text{diag}(\boldsymbol{m}_l) \cdot M_l$$

Then training with dropout $\approx$ minimising:

$$\min_{q(w)} \; \mathbb{E}_{w \sim q}[-\log p(\mathcal{D} \mid w)] + D_{\text{KL}}(q(w) \,\|\, p(w))$$

Practical upshot

Each dropout mask $\boldsymbol{m}^{(t)}$ is a **sample** from an implicit approximate posterior over weights.

MC averaging over T masks $\approx$ Bayesian model averaging.

Important caveats

- The variational family is **restrictive**
- Uncertainty quality depends on dropout rate p, placement, and T
- May underestimate or misrepresent uncertainty for some problems

MC Dropout: Exact Training Procedure

Training (regression with aleatoric noise)

1: Build network with **Dropout(p)** after each hidden layer
2: Output head: two scalars $(\hat{\mu}, \log \hat{\sigma}^2)$ per sample
3: **for** each mini-batch $\mathcal{B}$ **do**
4: Forward pass **(dropout is active by default)**
5: Compute heteroscedastic NLL:

$$\mathcal{L} = \frac{1}{B} \sum_{i \in \mathcal{B}} \left[\frac{(y_i - \hat{\mu}_i)^2}{2e^{s_i}} + \frac{s_i}{2} \right]$$

where $s_i = \log \hat{\sigma}_i^2$
6: Backprop + Adam/SGD on $\hat{w}$ (point estimate), **with L2 weight decay**
7: **end for**
8: **Same as a standard NN with dropout and weight decay;**
weight decay plays the role of $D_{\mathrm{KL}}(q(w) \,\|\, p(w))$ in the VI objective

MC Dropout: Full Inference Procedure

Algorithm (regression)

Training: standard dropout + NLL loss + L2 weight decay (no other changes)

Test-time inference:

for $t = 1, \dots, T$ **do**
  sample mask $\boldsymbol{m}^{(t)} \sim \mathrm{Ber}(1-p)^d$
  $\hat{y}^{(t)}, \hat{\sigma}^{2,(t)} \leftarrow f_{\hat{w}, \boldsymbol{m}^{(t)}}(x^*)$
end for
$\hat{\mu} \leftarrow \frac{1}{T} \sum_t \hat{y}^{(t)}$
$\hat{\sigma}^2_{\text{ep}} \leftarrow \frac{1}{T} \sum_t (\hat{y}^{(t)} - \hat{\mu})^2$
$\hat{\sigma}^2_{\text{al}} \leftarrow \frac{1}{T} \sum_t \hat{\sigma}^{2,(t)}$
$\hat{\sigma}^2_{\text{total}} \leftarrow \hat{\sigma}^2_{\text{ep}} + \hat{\sigma}^2_{\text{al}}$

PyTorch one-liner

Enable dropout at test:

```
model.eval()
for m in model.modules():
if isinstance(m, nn.Dropout):
m.train()
```

Then loop T forward passes.

Choosing T

$T \in [20, 100]$ is typically sufficient.
Plot variance-of-variance vs T to diagnose convergence.
T=50 is a sensible default.

The Three Methods Side by Side

	SGLD	**Bayes by Backprop**	**MC dropout**
Numbers stored per weight	1 (the current value)	2: μ_j and $\sigma_j = \mathrm{softplus}(\rho_j)$	1
What the update changes	the weight values themselves	μ_j and ρ_j, by gradient ascent on the ELBO	the weights, by ordinary SGD
Where the randomness is	noise added to each update	weights sampled, $w = \mu + \sigma \odot \varepsilon$	random dropout masks
How the posterior is represented	saved samples $w^{(s)}$	the distribution $q_\phi(w)$, an explicit Gaussian per weight	implicitly, by the masks
Prediction	average over the saved networks	sample $w \sim q_\phi$ several times and average	several passes with dropout on

Why VI (Bayes-by-Backprop) When MC Dropout Exists?

MC Dropout requires no architecture changes — so why would anyone do VI?

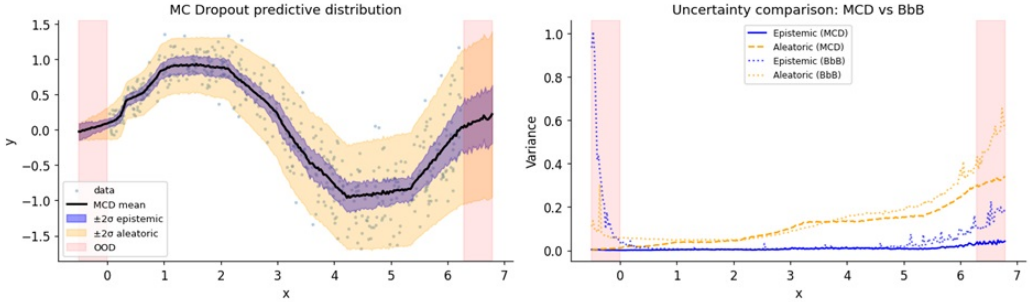
MC Dropout strengths

- **Zero training overhead**: retrofit onto any existing model
- **Same parameter count**: no memory increase
- **No special layers needed**
- **Works for any architecture**: CNNs, RNNs, Transformers
- Inference cost: $T\times$ one forward pass

MC Dropout limitations

- Restricted variational family: Bernoulli masks only
- Dropout rate p is a fixed hyperparameter — may be wrong
- Uncertainty quality depends sensitively on p and placement
- Cannot capture *weight correlations* across layers
- Often **underestimates epistemic uncertainty** on OOD data

MC Dropout Predictive distribution

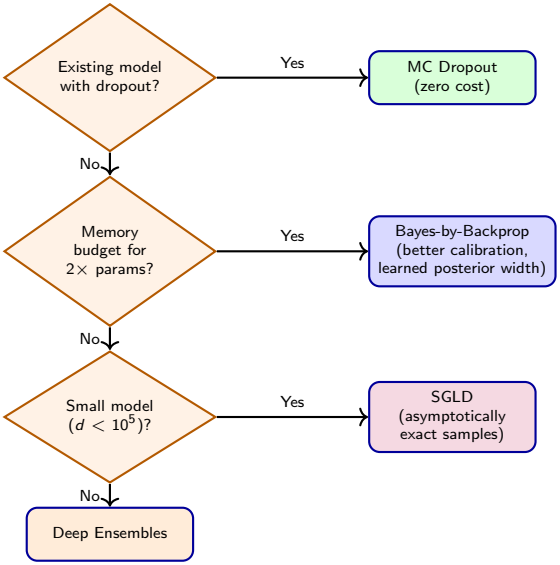


- **BbB epistemic** (blue dotted): large, peaks sharply at both OOD boundaries — *rich posterior*.
- **MCD epistemic** (blue solid): small, barely rises at OOD — *coarser posterior approximation*.
- **Both aleatoric curves** (orange): similar shape, growing rightward — consistent learned noise, as expected since both methods use the same heteroscedastic NLL loss.

Takeaway

MC Dropout trades *posterior richness* for *simplicity*: no extra parameters, no KL term, easy to retrofit to any existing network — but at the cost of less expressive uncertainty estimates, particularly for OOD detection.

MC Dropout vs. BbB vs. SGLD: Decision Guide



Uncertainty Quality: A Comparison

Property	MC Dropout	BbB	SGLD	Ensembles
Training cost	$1\times$	$1.5\times$	$1\times$ (long)	$M\times$
Parameter overhead	None	$2\times$	None (checkpoints)	$M\times$
Inference cost	$T\times$ fwd	$S\times$ fwd	$S\times$ fwd	$M\times$ fwd
Posterior quality	Approximate	Approximate	Asymptotic	Empirical
Calibration	Moderate	Good	Best	Good
OOD detection	Good	Good	Good	Better
Architecture changes	None	Yes (layers)	None	None
Easy to tune	Moderate	Hard	Hard	Easy

When MC Dropout fails

Long-range correlations in weight posterior (deep, narrow networks); very small dropout rates effectively give nearly-deterministic models.

Methods of Approximate Inference for BNNs

MCMC

HMC / NUTS / SG-MCMC

Define prior $p(\mathbf{w})$ and likelihood $p(\mathcal{D} \mid \mathbf{w})$

Run Markov chain targeting posterior $p(\mathbf{w} \mid \mathcal{D})$

Keep posterior samples $\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(S)}$

Predict at x^* using posterior samples

Average predictions + quantify spread (credible intervals)

Sampling-based (high fidelity, expensive)

Variational Inference

Bayes by Backprop

Choose family $q_\phi(\mathbf{w})$ (e.g., factorized Gaussian)

Optimise ELBO w.r.t. ϕ via gradients + reparam.

Obtain approximate posterior $q_{\phi^*}(\mathbf{w})$

Sample $\mathbf{w} \sim q_{\phi^*}(\mathbf{w})$ and predict on x^*

MC average + uncertainty from $q_{\phi^*}(\mathbf{w})$ samples

Optimisation-based (scalable, can be biased)

MC Dropout

Dropout as approx. Bayesian inference

Train network with dropout (+ weight decay)

At test time keep dropout ON

Run T stochastic forward passes with different dropout masks

Collect $\hat{y}_1, \ldots, \hat{y}_T$ for x^*

Estimate predictive mean and variance from samples

Practical stochastic approx. (easy to retrofit to NNs)

Target quantity: predictive distribution $p(y^* \mid x^*, \mathcal{D})$

Diagnosing Approximate Inference: Did It Work?

Approximate inference can *look* plausible while being systematically wrong.

Five mandatory sanity checks

1. **Epistemic uncertainty shrinks with data:** Hold out x^*; retrain with more data nearby; verify $\hat{\sigma}^2_{\text{ep}}(x^*)$ decreases.
2. **Uncertainty grows away from data:** Evaluate on OOD inputs (e.g. unseen arrhythmia class); epistemic uncertainty should be higher than in-distribution.
3. **Posterior samples are diverse:** Plot T stochastic forward passes; they should differ visibly (if they collapse to one line, the posterior has collapsed).
4. **Prior sensitivity:** Try $\tau \in \{0.1, 1.0, 10.0\}$; results should be qualitatively robust.
5. **NLL on held-out set:** Better UQ method $\equiv$ lower predictive NLL (a proper scoring rule; accuracy alone is not sufficient).

Summary

Implementation take-aways

1. **Training**: use heteroscedastic NLL loss with two output heads $(\hat{\mu}, \log \hat{\sigma}^2)$ to learn both epistemic (via MC dropout/BbB) and aleatoric (via learned variance) uncertainty.
2. **SGLD**: inject Gaussian noise proportional to $\sqrt{\epsilon_t}$ into SGD updates; discard burn-in; asymptotically exact but expensive.
3. **Bayes-by-Backprop**: store $(\boldsymbol{\mu}, \boldsymbol{\rho})$ per weight; reparameterise; maximise stochastic ELBO with analytic KL; doubles parameter count.
4. **MC Dropout**: cheapest retrofit; keep dropout on at test time with `model.eval()+enable_dropout()`; $T = 50$ passes; freeze BatchNorm.
5. **Choice rule**: MCD first; BbB if you can redesign; SGLD for benchmarks; Ensembles for safety-critical.
6. **Biomedical applications**: ECG OOD arrhythmia detection, PPG SpO_2 noise decomposition, EEG active annotation, EMG sensor drift alerting.

Key References

- **Gal & Ghahramani (2016):** "Dropout as a Bayesian Approximation" — MC Dropout theory.
- **Blundell et al. (2015):** "Weight Uncertainty in Neural Networks" — Bayes-by-Backprop.
- **Kendall & Gal (2017):** "What Uncertainties Do We Need?" — epistemic/aleatoric decomposition.
- **Welling & Teh (2011):** "Bayesian Learning via Stochastic Gradient Langevin Dynamics" — SGLD.
- **Lakshminarayanan et al. (2017):** "Simple and Scalable Predictive Uncertainty using Deep Ensembles."
- **Zhang et al. (2020):** "Cyclical Stochastic Gradient MCMC for Bayesian Deep Learning."
- **Ovadia et al. (2019):** "Can You Trust Your Model's Uncertainty?" — benchmarks under dataset shift.
- **Gawlikowski et al. (2023):** "A Survey of Uncertainty in Deep Neural Networks."

Questions?